

AI-Powered Medical Education Content Factory

A production two-stage pipeline that ingests medical literature and autonomously generates structured study plans — concepts, MCQs, clinical cases, and verified assessments — at scale.

[Python] [Azure Functions] [Azure AI Search] [GPT-4o] [PubMedBERT] [Azure SQL] [Blob Storage] [Queue-driven orchestration]

01 — OVERVIEW

What Was Built

This system is a fully automated medical content pipeline. You feed it a collection of medical PDFs — textbooks, clinical guidelines, journal articles — and it outputs a complete, structured, pedagogically sound study plan: teaching summaries, multiple-choice questions with rationales and variants, clinical case vignettes, and their associated assessment MCQs.

Nothing in the output is copied verbatim. Every concept, question, and case is **synthesized by GPT-4o under tight constraints**, grounded in the source literature, and validated by multiple automated quality gates before being committed to the database.

"The system does not re-process documents at runtime. RAG ingestion materialises a searchable, reference-rich, hierarchically labelled corpus once — the study plan pipeline then queries that corpus on demand, keeping content generation fast and cost-efficient."

2	8	768-D	12+	5-Layer
DECOUPLED PIPELINE STAGES	ASYNC AZURE FUNCTIONS	BIOMEDICAL EMBEDDING VECTORS	SQL TABLES IN SCHEMA	HALLUCINATION CONTROLS

02 — ARCHITECTURE

Two-Stage Pipeline

The system is intentionally split into two independent stages with a shared contract — an Azure AI Search index — as the only coupling point between them. This makes each stage independently testable, scalable,

and replaceable.

01 RAG Pipeline — Corpus Preparation (Offline / Batch)

Ingests raw medical PDFs, detects hierarchical headings (Topic → Subtopic → Sub-subtopic), splits content into semantically meaningful chunks (≤ 2000 chars), enriches each chunk with GPT-4o metadata (summaries, objectives, complexity scores, key facts), embeds using **PubMedBERT** (768-D CLS vectors), and uploads all vectors and metadata to **Azure AI Search**. Source PDFs and manifests are stored in **Azure Blob Storage**.

```
[stage1_chunk_pdf.py] [stage2_meta_tag.py] [stage3_vector_index.py] [pdfminer] [ReportLab]
[HuggingFace transformers]
```

02 Study Plan Pipeline — Content Generation (Event-Driven)

Eight **Azure Functions** daisy-chained through **Storage Queues** consume the search index to generate complete CME study plans. Each function has a single responsibility: subtopic discovery, reference harvesting, concept synthesis, MCQ generation, clinical case creation, case MCQ authoring, bundle verification, and final plan assembly — all written to Azure SQL.

```
[Azure Functions v4] [Storage Queues] [Azure SQL (ODBC)] [GPT-4o JSON mode] [OData filters]
```

→ Shared Contract: Azure AI Search Index

The two stages share no code. They are coupled only through a well-defined Search document schema: topic, subtopic, sub_subtopic, content, references[], embedding[768], heading_path. This design means the corpus can be re-indexed independently without touching any study plan code.

03 — ORCHESTRATION

Event-Driven Queue Topology

The study plan pipeline uses eight Azure Storage Queues to fan out work across independently scaling function workers. A single HTTP call to POST /api/enqueueTopic seeds the entire chain.



enqueueTopic (HTTP)

Entry point. Inserts topic + placeholder subtopics into SQL, fires topic-queue.



generateSubtopics

Queries Search index to discover hierarchical subtopic outline; detects vignette sections; populates SQL rows.



harvestReferences

Extracts, deduplicates, and persists bibliographic references from index chunks via OData filter.



generateConcept

Stitches chunk text, prompts GPT-4o for bullet-point teaching summaries, runs Jaccard duplicate guard.



generateMcq

Two-step planner → generator for MCQs with distractors, rationales, variants, and structural validation loops.



generateCase / generateCaseMcq

Synthesises clinical vignettes from source content, then authors case-specific MCQs grounded in the vignette.



verifyCaseBundle

LLM-as-judge: GPT-4o cross-checks case MCQs against the vignette and concept, returning a structured verdict.



assemblePlan

Waits for all subtopics to reach terminal state, deduplicates, gap-detects, then MERGES final JSON into Azure SQL.

04 — AI ENGINEERING

LLM Orchestration Patterns

The system uses GPT-4o throughout, but with carefully engineered prompting patterns to maintain output quality, structural integrity, and factual grounding. This is not simple prompt-and-parse — it is a multi-layer LLM orchestration system.

JSON Response Format

All LLM calls use `response_format={"type":"json_object"}` with structured schema enforcement via post-processing. Guarantees machine-parseable output at every stage.

Planner → Generator (MCQ)

MCQ generation is split into two LLM passes: first a planner that decides question topics and learning objectives, then a generator that authors the actual questions — separating intent from execution.

No-New-Facts Guardrail

Concept and case MCQ prompts explicitly instruct the model to reason only from the provided SOURCE text, not from parametric knowledge — reducing hallucination risk for clinical content.

LLM-as-Judge Verification

`verifyCaseBundle` uses a separate GPT-4o call to audit whether case MCQs are internally consistent with the vignette and concept, returning a structured pass/fail verdict with reasoning.

Self-Heal / Second
Pass

Both Stage 2 and generateConcept detect schema-light or clipped outputs and issue a targeted repair prompt, recovering gracefully without full regeneration.

Lexical Overlap
Linting

Generated concepts are checked for minimum keyword hit count from the source subtopic title — if the output doesn't mention enough topic-relevant terms, it is rejected and regenerated.

5-Gram Jaccard
Deduplication

Sibling concepts within a topic are compared using shingling similarity. Concepts above a threshold trigger a disambiguation re-prompt rather than silent acceptance of near-duplicate content.

05 — RETRIEVAL DESIGN

How Retrieval Works

The RAG approach here is deliberately different from standard semantic similarity search. Rather than top-k nearest-neighbour queries, the system performs **full-corpus retrieval per subtopic using OData filters** on the hierarchical metadata fields.

```
# harvestReferences – OData filter example
filter = (
  f"topic eq '{topic_name}' and subtopic eq '{subtopic_title}'"
)
results = search_client.search(
  search_text="", # full-corpus fetch, not semantic
  filter=filter,
  select=["references", "sequence", "chunk_index"],
  order_by=["sequence asc", "chunk_index asc"]
)
```

→ Chunks are retrieved in **deterministic reading order** (sequence → heading_path → chunk_index), preserving narrative flow for concept synthesis.

→ Coverage estimation uses **keyword search as a fallback signal** when aggregated char_count for a subtopic is zero — allowing the system to detect genuinely sparse topics before investing LLM calls.

→ Embeddings (PubMedBERT 768-D CLS) are stored in the same index and available for semantic search use cases, but the primary pipeline uses structured filtering for reproducibility.

→ References are deduplicated at harvest time via SHA-256 hashing of normalised citation strings before SQL insert.

SQL Schema & Output Shape

The assembled study plan is built from a normalised Azure SQL schema (schema cme) with careful status-machine management across subtopic rows. The final output is a rich, nested JSON blob MERGED into `cme.study_plans`.

cme.topics

Root entity. UUID keyed. One row per medical topic ingested.

cme.subtopics

Status machine: `queued` → `refs_pending` → `concept_pending` → `mcq_ready` → (case states).
Central coordination table.

cme.concepts

GPT-generated markdown bullet summaries, one per subtopic.
Token count tracked for downstream quality gates.

cme.questions + choices + variants

MCQs with full distractor rationales and paraphrased stem variants for reuse.

cme.cases

Clinical vignette text with GPT-extracted case metadata and verification status.

cme.references + subtopic_refs

Normalised bibliography.
Deduped by source hash.
Many-to-many link to subtopics.

cme.qa_reviews

Stores LLM-as-judge verdicts for MCQ plans and case bundles.
Full audit trail.

cme.study_plans

Final MERGE target. Nested JSON: `subtopics` → `concepts` → `MCQs` → `cases` → `references`.

cme.fail_log + cme.content_gaps

Structured failure log and coverage gap tracking for observability and re-processing.

Resilience & Error Handling

The pipeline is designed to survive partial failures without losing state. Every stage has explicit fallback, retry, or graceful degradation behaviour.

→ **Exponential backoff** on Azure Search uploads and Blob GET operations (Stage 3), with configurable `MAX_RETRIES` and `BASE_SLEEP`.

→ **GPT retry loops** in Stage 2 (meta tagging) and MCQ generation — up to 4 validation passes before marking a failure in `cme.fail_log`.

→ **Fallback metadata** on GPT exhaustion in Stage 2 — rather than crashing the batch, a schema-safe fallback object is written so the chunk still enters the index.

→ **Queue visibility extension** in `assemblePlan` — when a topic isn't fully done yet, the plan queue message is extended (not re-queued) to avoid duplicate assembly.

→ **Case path wrapped in try/except** — if case generation fails for a subtopic, it degrades to a non-case study plan entry rather than blocking the entire topic.

→ **Application Insights** sampling configured in host.json for distributed function tracing.

08 — INFRASTRUCTURE

Technology Stack

Azure AI Search

Vector + metadata index. OData-filtered retrieval.
Shared contract between both pipeline stages.

Azure OpenAI (GPT-4o)

All LLM calls — metadata extraction, concept synthesis, MCQ generation, case authoring, verification.

Azure Functions (Python)

8 serverless workers. Queue triggers + HTTP trigger.
Extension bundle v4.

Azure Storage Queues

Async fan-out/fan-in orchestration. Raw JSON encoding (no base64 wrapping).

Azure SQL (ODBC)

All structured state: topics, subtopics, concepts, MCQs, cases, references, audit logs.

Azure Blob Storage

Stores per-chunk PDFs, ingestion manifests, and meta JSON artifacts.

PubMedBERT (HuggingFace)

microsoft/BiomedNLP-PubMedBERT-base-uncased-abstract. 768-D CLS embeddings for biomedical domain accuracy.

pdfminer · ReportLab

PDF text extraction and per-chunk PDF rendering in the RAG batch pipeline.

Built end-to-end: RAG pipeline · Azure Functions orchestration · LLM prompt engineering · SQL schema design · Reliability patterns

This project represents the full design, implementation, and architectural decisions of an AI-native content generation system.